

M1-MIAGE

TER-
PHP

Référence utilisée
Fabien Torre

1-Instructions, variables, types

Instructions

Un programme PHP est encadré par les symboles `<?php` et `?>`, avant et après peuvent se trouver des contenus textuels ou HTML. Un processeur aura la charge d'exécuter le code PHP et de le remplacer par le résultat de l'exécution. Dans le cas d'une application web, cette tâche incombera au serveur web.

En PHP, les blocs d'instructions sont délimités par des accolades : `{` est l'équivalent d'un *Début*, `}` d'une *Fin*.

Chaque instruction est impérativement suivie d'un point-virgule (`;`).

Types de variables

Même si aucune déclaration des variables et de leurs types n'est nécessaire, il existe en PHP trois types de données : les types *scalaires*, les tableaux *classiques* et les tableaux *associatifs*.

Les types *scalaires* regroupent les types de base : entiers, réels, caractères et chaîne de caractères, etc.

En PHP, toutes les variables commencent par le signe \$.

Deux symboles à distinguer très vite :

- + symbole `=` pour l'affectation ;
- + symbole `==` pour le test d'égalité.

```
$x = 2;  
$y = 3.14;  
$z = 'toto';
```

Comme son nom l'indique, une constante une fois définie ne changera plus de valeur. Elles sont donc bien distinguées des variables : elles ne commencent pas par le signe \$ et sont définies à l'aide la fonction `define`.

```
define('NOM','toto');
```

Opérateurs

Nous disposons en PHP des opérateurs arithmétiques classiques : `+`, `-`, `*`, `/` et de l'opérateur de concaténation entre chaînes de caractères noté par un point.

```
$i = 10;  
$i = $i + 1;  
$i = $i + 9;  
$texte = 'La variable i vaut '.$i;  
$texte = $texte."\n";
```

Remarquez ici que PHP n'a pas de contrôle fort sur les types de variables : une même variable peut subir une opération arithmétique puis une opération propre aux chaînes de caractères sans provoquer d'erreur. Au final, la variable `$texte` contient donc *La variable i vaut 20*.

Il existe d'autres manières souvent pratiques d'exprimer ces opérations. Par exemple, les opérations ci-dessus sont strictement équivalentes à :

```
$i = 10;  
$i++;  
$i += 9;  
$texte = 'La variable i vaut '.$i;  
$texte .= "\n";
```

Opérateurs booléens : `==`, `!=`, `>=`, `<=`, `&&`, `||`, `!`, etc.

Chaînes de caractères et instruction de sortie

L'affichage est réalisé à l'aide l'instruction **echo**. Cette instruction est particulièrement importante : c'est elle qui va produire le texte et les balises HTML du document final.

En PHP, les chaînes de caractères sont délimitées soit par des apostrophes, soit par des guillemets. Dans le premier cas, le contenu de la chaîne n'est pas interprété par PHP, dans le second une variable apparaissant dans la chaîne est remplacée par sa valeur.

```
$t = 'coucou';  
$v = 5;  
echo '$v fois $t\n';  
echo "$v fois $t\n";
```

Dans ce cas, le premier affichage produira *5 fois coucou*, le second lui affichera *5 fois coucou* avec un passage à la ligne ensuite (codé par `\n`).

2-Procédures et fonctions en PHP

Définition

```
function nom_de_la_fonction (paramètres) {  
    // des instructions ici  
}
```

```
function EntetePage ($titre,$fond='pink') {  
    // corps de la fonction ici  
}
```

```
function EntetePage ($titre,$fond) {  
    echo "<HTML>\n";  
    echo "<HEAD>\n";  
    echo "<TITLE>$titre</TITLE>\n";  
    echo "</HEAD>\n";  
    echo "<BODY BGCOLOR=\"\$fond\">\n";  
    echo "<H1>$titre</H1>\n";  
}
```

Notons qu'en PHP les paramètres sont passés *par valeur*, c'est-à-dire que les procédures travaillent sur une *copie* des objets passés en paramètres. Si les objets donnés à une procédure sont susceptibles d'être modifiés et si nous souhaitons que ces modifications soient visibles de l'extérieur de la procédure, le passage doit se faire *par référence*, avec la syntaxe suivante :

```
function EntetePage (&$titre) { // notez le symbole &  
    // corps de la fonction ici, qui peut modifier le titre  
}
```

Faire appel

```
Entetepage('Ma première page','blue');
```

```
Entetepage($titre_page,$couleur_de_fond);
```

```
Entetepage($titre_page);
```

Bibliothèque de fonctions

Une manière élégante de programmer consiste à rassembler dans un même fichier PHP toutes les fonctions d'usage courant. Les différentes pages PHP peuvent ensuite utiliser ces fonctions en faisant appel à la bibliothèque de fonctions en utilisant la fonction *require*.

Ainsi, si l'on imagine que la fonction *Entetepage* se trouve dans un fichier nommé `bibli.php`, on pourra écrire dans un nouveau fichier :

```
require('bibli.php');  
Entetepage('Ma première utilisation de bibliothèque');
```

Structures de contrôles

Modulo les conventions PHP, on retrouve les structures de contrôle classiques. Le *si alors sinon* :

```
if (une condition ici) {  
    // des instructions ici  
} else {  
    // des instructions ici  
}
```

La boucle *tant que* :

```
while (une condition ici) {  
    // des instructions ici  
}
```

Et la boucle *pour* :

```
for ($i=0 ; $i<10 ; $i++) {  
    // des instructions ici  
}
```

Les tableaux

Deux types de tableaux :

- + les tableaux *classiques* : pas de contrainte sur le contenu des cases, celles-ci sont numérotées à partir de 0 ;
- + les tableaux *associatifs* dont la particularité est que les cases ne sont plus indicées par un numéro mais par un texte.

Manipulation des tableaux

Tout d'abord l'affectation de valeurs aux cases des tableaux.

```
// définition d'un tableau vide et remplissage des cases
$t = array();
$t[0] = 3.14;
$t[1] = 'coucou';
$t[] = 10;

// syntaxe équivalente
$t = array(3.14, 'coucou', 10);
```

Cette dernière notation permet d'insérer une valeur dans la première case disponible (ici, la troisième) sans pour autant connaître la taille du tableau. À noter que cette taille peut être obtenue par la fonction `sizeof`.

Enfin, pour définir les tableaux associatifs, nous disposons de deux syntaxes :

```
$fiche1 = array();
$fiche1['prenom'] = 'Fabien';
$fiche1['nom'] = 'Torre';
$fiche1['mail'] = 'torre@univ.fr';
$fiche1['url'] = 'http://www.univ.fr/~toto/';
echo 'Mail de '.$fiche1['prenom'].' : '.$fiche1['mail'];

$fiche2 = array(
    'prenom' => 'Fabien',
    'nom' => 'Torre',
    'mail' => 'torre@univ.fr',
    'url' => 'http://www.univ.fr/~toto/',
);
echo 'Mail de '.$fiche2['prenom'].' : '.$fiche2['mail'];
```

Parcours des tableaux : boucle “Foreach”

Enfin, la boucle *foreach* est dédiée au parcours de tableaux, simples ou associatifs :

```
foreach ($tab as $val) {  
    // des instructions ici portant sur $val  
}  
  
foreach ($tab as $clef => $val) {  
    // des instructions ici portant sur $clef et $val  
}
```

Web dynamique et PHP

La particularité des programmes PHP est qu'ils peuvent se trouver dans une page web, au milieu du code HTML : lorsque une telle page est demandée au serveur, celui-ci exécute le programme PHP qui s'y trouve et fournit le résultat.

Cela amène de nouvelles contraintes :

- + une telle page devra avoir l'extension `.php` pour indiquer au serveur qu'il y a du code à interpréter à l'intérieur ;
- + il faudra impérativement disposer d'un serveur web pour visualiser les pages PHP.

Du côté utilisateur par contre, le processus est totalement transparent : celui-ci ne recevra pas le code PHP mais un simple code HTML dont il ne pourra déterminer s'il a été écrit à la main ou généré par un programme.

Pour réaliser cette intégration on utilise une nouvelle balise pour signaler les blocs de code PHP. Le schéma général sera le suivant.

```
avant : du code HTML
<?php
    ici du code PHP !
    ici du code PHP !
    ici du code PHP !
?>
après : du code HTML
```

Transmission des paramètres

On regarde ici comment faire passer des valeurs à un script PHP.

URL

Une première méthode consiste à faire passer ces valeurs sur dans l'adresse du script PHP. Par exemple, on pourra écrire :

```
http://www.grappa.univ-lille3.fr/~torre/guide.php?id=coursphp
```

Dans `guide.php`, on disposera alors d'une variable `$id` qui contiendra ici la valeur `coursphp`. Il est possible de faire passer plusieurs valeurs en les séparant par le signe `&` :

```
http://www.grappa.univ-lille3.fr/~torre/guide.php?id=coursphp&lang=fr
```

Formulaires HTML

Cette fois les valeurs nous sont fournies par un utilisateur à travers un formulaire donc l'ACTION pointe sur un script PHP et la METHOD indique la valeur `POST`.

Ce script PHP pourra utiliser les valeurs venant du formulaire à travers des variables dont les noms correspondront aux NAME indiqués dans le formulaire.

Pour une question autorisant les réponses multiples, le NAME devra se terminer par des `[]`, ce qui permettra dans le script PHP de manipuler une variable de type tableau.